

DLoader: Migration of Data from SQL to NoSQL Databases

Checkpoint Report - NoSQL Cloud Datastores: Mongoose & MongoDB vs NodeJS

Student	Taher Amine ELHOUARI
Scope	SQL-to-NoSQL migration, DLoader architecture, transformation, performance, integrity, migration planning, and case studies
Core source	Rajaram, Sharma & Selvakumar, DLoader: Migration of Data from SQL to NoSQL Databases, Springer, 2023

Important framing: DLoader in this checkpoint refers to the academic migration framework proposed by Rajaram, Sharma and Selvakumar. It is not the same product as MongoDB Relational Migrator. The report treats the paper as the controlling source for DLoader and uses current MongoDB material only for modern comparison and industry examples.

1. Introduction to Data Migration

What is data migration and why is it important?

Data migration is the controlled movement of data from one storage technology, schema, or platform to another. It matters because applications and organizations often outgrow an existing data architecture. A migration must preserve usable data while adapting it to the destination model, minimizing data loss, corruption, downtime, and application disruption.

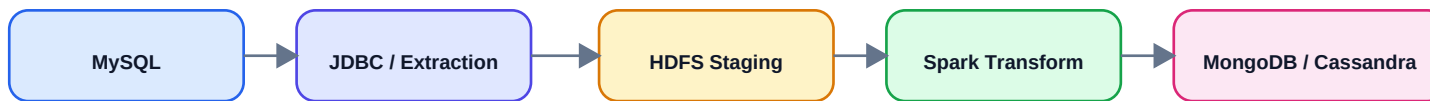
Key SQL vs NoSQL differences

Aspect	SQL / relational	NoSQL
Structure	Tables, rows, fixed columns	Documents, key-value, wide-column, graph, etc.
Schema	Usually predefined and normalized	Often flexible and application-oriented
Relationships	Primary/foreign keys and joins	Often embedding, denormalization, or application-side relationships
Scaling	Often scale-up plus replicas/sharding depending on engine	Frequently designed for horizontal scaling
Design focus	Data integrity and normalized relationships	Access patterns, distribution, flexibility, scale

The central migration difficulty is therefore not simply copying rows. The relational structure must often be **reshaped** so that the destination NoSQL database can serve the application's expected queries without relying on relational joins.

2. Overview of DLoader

DLoader high-level migration pipeline



Extract -> stage -> standardize/denormalize/map -> load

DLoader is a proposed framework for migrating data from a relational database such as **MySQL** to NoSQL targets such as **MongoDB** and **Cassandra**. Its architecture extracts source data, stages it in Hadoop Distributed File System (HDFS), processes it with Apache Spark/Spark SQL, transforms and standardizes it for the target model, then loads the transformed data into the selected NoSQL database.

Capability	How DLoader uses it
Source extraction	Uses JDBC to read MySQL data; timestamps can identify recently added data.
Staging layer	Uses an HDFS cluster as an intermediate layer before transformation.
Processing	Loads source tables into Spark SQL data frames.
Transformation	Renames columns, converts data types, adds/removes fields, standardizes values, and denormalizes relational structures.
Mapping	Maps source table columns to destination NoSQL fields/columns.
Targets	The paper demonstrates migration to MongoDB and Cassandra.
Performance mechanism	Uses Spark's parallel processing and clustered storage/compute.

3. Migration Process

Step	Action
1. Assess	Inventory source tables, keys, data types, volumes, quality problems, and application queries.
2. Design target	Choose MongoDB/Cassandra structures based on target access patterns; decide what to embed or denormalize.
3. Connect	Use JDBC to connect DLoader/Spark to the MySQL source.
4. Extract	Read source data; use timestamps when incremental/delta extraction is needed.
5. Stage	Place relational data in HDFS while initially preserving source structure.
6. Transform	Load staged tables into Spark data frames; standardize values, change types/names, add/remove fields, and denormalize joins.
7. Map	Map transformed source columns into destination NoSQL fields.
8. Load	Write transformed data into MongoDB or Cassandra.
9. Validate	Compare source and target counts, sampled records, keys, totals, null handling, and business rules.
10. Cut over	Freeze or synchronize final changes, switch the application, monitor, and retain rollback options.

Common challenges and mitigations

Challenge	Mitigation
Schema mismatch	Design the target around access patterns rather than copying normalized tables literally.
Foreign-key relationships	Embed related data or duplicate selected fields where appropriate.
Data type incompatibility	Define explicit type mappings and test edge cases.
Dirty/inconsistent source data	Standardize names, dates, codes, phone numbers, nulls, and duplicates before loading.
Large volume	Partition work, use parallel Spark processing, and migrate in controlled batches.
Source changes during migration	Use timestamp/delta extraction and a controlled final synchronization window.
Application compatibility	Test application queries against the new NoSQL model before cutover.

4. Data Transformation

DLoader transforms relational data inside Spark SQL data frames with the **target schema in mind**. The paper specifically describes renaming columns, changing data types, adding or removing columns, standardizing formats, and denormalizing relational tables because the destination NoSQL model should not depend on traditional joins.

SQL source	Possible NoSQL transformation
customers(id, name) + addresses(customer_id, city)	MongoDB customer document embeds address fields or an address subdocument.
orders + order_items	One order document contains an embedded array of line items.
gender values: M, Male, male	Standardize to one representation such as 'M' or 'male'.
date stored in multiple text formats	Normalize to a consistent date/timestamp type.
phone values with spaces/dashes/country-code differences	Normalize into a consistent canonical phone representation.
SQL DECIMAL / integer / VARCHAR	Map each source type to the appropriate BSON or Cassandra data type.

5. Performance Considerations

Migration performance depends on source read throughput, network bandwidth, HDFS/Spark cluster capacity, transformation complexity, destination write throughput, indexes, document size, and the amount of data being moved. Large denormalizing joins can be especially expensive because they increase CPU, memory, shuffle, and network activity.

DLoader's performance-oriented design relies primarily on **Spark parallel processing** and clustered HDFS staging. The paper's experiment used a Hadoop/Spark testbed to migrate generated immunization data from MySQL to Cassandra and MongoDB. Its reported results found Cassandra to require less storage and to provide better performance than MongoDB for the tested workload; this is an experimental result for that dataset, not a universal rule.

Practical optimization: benchmark with representative data, partition extraction/load work, avoid unnecessary transformations, size the cluster for peak migration load, and test destination indexes after the load.

6. Consistency and Integrity

The DLoader paper supports integrity indirectly by staging extracted data, preserving the source structure before transformation, applying explicit transformation/mapping rules, and using timestamps to identify newly added data. However, the paper does **not** describe a built-in end-to-end reconciliation engine that automatically proves that every source record and transformed value matches the target.

Therefore, a production migration plan should add explicit verification controls: row/document counts, primary-key coverage, duplicate detection, null checks, field-level samples, aggregate totals, checksums/hashes for stable fields, and application-level business-rule tests. For financial or regulated data, reconciliation reports and exception handling should be retained as migration evidence.

Verification	Example
Count reconciliation	Source 2,000,000 rows -> expected 2,000,000 target root documents unless mapping intentionally changes cardinality.
Key reconciliation	Every source customer_id appears exactly once in the expected target representation.
Aggregate checks	SUM(order_total), counts by status, counts by month/branch.
Field sampling	Compare representative source records to transformed target documents.
Business rules	No negative age, valid status values, required customer references present.
Application testing	Run critical read/write workflows against the migrated dataset.

Modern comparison: MongoDB Relational Migrator has an optional data-verification process for one-time migration jobs that checks source/target counts and record equivalence. That capability belongs to MongoDB Relational Migrator, not to the DLoader paper.

7. Practical Application - Simple DLoader Migration Plan

Scenario: a MySQL e-commerce database contains Customers, Orders, OrderItems, Products, and Categories. The target is MongoDB because the application frequently reads an order together with its items.

Phase	Implementation
Discover	Profile tables, sizes, relationships, nulls, duplicates, and critical queries.
Target model	Create customers, products, and orders collections; embed OrderItems inside each order document.
Connect/extract	Configure JDBC access to MySQL and extract tables to the HDFS staging layer.
Transform	Use Spark SQL joins while the data is still relational/staged; build embedded item arrays; convert data types; standardize dates and status values.
Map/load	Map transformed frames to MongoDB collections and load in controlled batches.
Validate	Compare record counts, order totals, customer coverage, sampled documents, and business-rule aggregates.
Delta/final sync	Use timestamps to capture records added since the initial extraction; briefly control writes during final cutover if necessary.

Phase	Implementation
Cutover	Switch application connection/configuration, monitor errors and latency, retain the source database until acceptance criteria are met.

8. Case Studies and Examples

DLoader research experiment

The DLoader paper demonstrates its framework on an immunization dataset representing children, mothers, and immunization details. The source data was held in MySQL, staged and processed with Hadoop/Spark, then migrated to Cassandra and MongoDB. The lesson is that relational-to-NoSQL migration requires both **schema transformation** and **performance testing**; choosing a NoSQL target should be driven by the intended workload rather than by the label 'NoSQL' alone.

Industry example - Nationwide Building Society

MongoDB documents Nationwide's use of Relational Migrator to move data from SQL to MongoDB. The published case study reports that the team valued schema visualization and reduced migration effort. The lesson for a DLoader-style project is to make schema mapping visible, test mappings iteratively, and treat migration tooling as a way to reduce manual conversion risk rather than bypass data-model design.

Industry example - Powerledger

Powerledger moved a large-scale energy platform from a previous relational architecture toward MongoDB Atlas, including years of data and many microservices. Its case study emphasizes scalability, security, testing, and avoiding service disruption. The lesson is to migrate incrementally, validate performance at realistic scale, and make operational continuity a first-class migration requirement.

9. Conclusion

Advantages of SQL -> NoSQL	Disadvantages / risks
Flexible document or wide-column structures	Requires data-model redesign and application changes
Horizontal scale for suitable workloads	Denormalization can duplicate data
Can align storage with application access patterns	Loss of familiar joins/relational constraints may move responsibility to application logic
Can improve performance for targeted read/write patterns	Wrong target model can perform worse than the original
Supports large and varied datasets	Migration, validation, cutover, and rollback add operational complexity

I would recommend DLoader when an organization has a **MySQL relational source**, a well-understood target schema in **MongoDB or Cassandra**, large enough data volumes to benefit from Spark/HDFS parallel processing, and a need for repeatable extraction, transformation, standardization, and mapping. I would not choose it blindly for a small migration or where a supported production migration product already provides stronger operational tooling, verification, monitoring, and support.

Final conclusion: the most important lesson is that SQL-to-NoSQL migration is a data-model transformation project, not merely a copy operation. DLoader provides a useful academic architecture for extraction, staging, transformation, mapping, and loading, but a production implementation should add rigorous reconciliation, rollback, security, observability, and cutover controls.

References

Rajaram, K., Sharma, P., & Selvakumar, S. (2023). DLoader: Migration of Data from SQL to NoSQL Databases. In *Proceedings of the International Conference on Cognitive and Intelligent Computing (ICCIC 2021)*, Volume 2, pp. 193-204. Springer Nature Singapore. DOI: 10.1007/978-981-19-2358-6_19.

MongoDB Documentation. Relational Migrator - Data Modeling, Migration Jobs, and Data Verification. Used only as a modern comparison, not as evidence of DLoader features.

MongoDB customer case studies. Nationwide Building Society and Powerledger. Used as current industry examples of relational-to-document migration.